

Cheatsheet

Useful commands and tips

Variables and Types

Variables

- **Definition:** Containers for storing information.
- **Example:** `x = 10`

Data Types

- **Integers (int):** Whole numbers (e.g., count of dates).
- **Floats (float):** Decimal numbers (e.g., compatibility score).
- **Booleans (bool):** True/False values (e.g., availability).
- **Strings (str):** Text values (e.g., names).

```
name = "Alexander" # String variable
flags = 0           # Integer variable
butterflies = True  # Boolean variable
```

Type Conversion

- **Checking:** Use `type()` to check the type of a variable.
- **Conversion:**
 - `int()`: Converts to integer.
 - `float()`: Converts to float.
 - `str()`: Converts to string.
 - `bool()`: Converts to boolean.

String Formatting

- **Concatenation:** Combine strings using `+`.
- **Formatting:** Use `f"..."` for formatted strings.

```
name = "Alexander"
print(f"Hello, {name}!")
```

```
Hello, Alexander!
```

Comparisons

Comparison Operators

Sym- bol	Meaning	Example
==	Equal to	score == 100
!=	Not equal to	degree != "Computer Science"
<	Less than	salary < 80000
>	Greater than	experience > 5
<=	Less than or equal to	age <= 65
>=	Greater than or equal to	test_score >= 80

Logical Operators

Sym- bol	Meaning	Example
and	Both conditions must be true	score > 80 and experience > 5
or	At least one condition must be true	score > 80 or experience > 5
not	Condition must be false	not (score > 80)

Decision-Making

if Statements

- **Structure:**

```
if condition:  
    # code to execute if condition is True
```

- **Example:**

```
flat_rating = 8  
if flat_rating >= 7:  
    print("This is a good apartment!")
```

```
This is a good apartment!
```

if-else Statements

- **Structure:**

```
if condition:
    # code to execute if condition is True
else:
    # code to execute if condition is False
```

- **Example:**

```
flat_rating = 4
if flat_rating >= 7:
    print("Apply for this flat!")
else:
    print("Keep searching!")
```

```
Keep searching!
```

if-elif-else Statements

- **Structure:**

```
if condition:
    # code to execute if condition is True
elif condition:
    # code to execute if condition is False
else:
    # code to execute if condition is False
```

- **Example:**

```
flat_rating = 8
if flat_rating >= 9:
    print("Amazing flat - apply immediately!")
elif flat_rating >= 7:
    print("Good flat - consider applying")
else:
    print("Keep looking")
```

```
Good flat - consider applying
```

Complex Conditions

- **Nested if Statements:** Use if statements inside other if statements.
- **Logical Operators:** Combine conditions using `and`, `or`, `not`.
- **Structure:**

```
if (condition1) and (condition2):
    # code if both conditions are True
elif (condition1) or (condition2):
```

```
# code if at least one condition is True
else:
    # code if none of the conditions are True
```

- **Example:**

```
flat_rating = 9
price = 900
if (flat_rating >= 9) and (price < 1000):
    print("Amazing flat - apply immediately!")
```

```
Amazing flat - apply immediately!
```

Lists and Tuples

Lists

- **Definition:** Ordered, mutable collections of items.
- **Creation:** Use square brackets `[]`.

```
ratings = [4.5, 3.8, 4.2]
restaurants = ["Magic Place", "Sushi Bar", "Coffee Shop"]
```

Accessing Elements

- **Indexing:** Use `[index]` to access elements.

```
print(restaurants[0]) # Access the first element
```

```
Magic Place
```

- **Negative Indexing:** Use `[-1]` to access the last element.

```
print(restaurants[-1]) # Access the last element
```

```
Coffee Shop
```

- **Slicing:** Use `[start:end]` to access a range of elements.

```
print(restaurants[0:2]) # Access the first two elements
```

```
['Magic Place', 'Sushi Bar']
```

Adding Elements

- **Appending:** Use `append()` to add an element to the end of the list.

```
restaurants.append("Pasta Place")
```

- **Inserting:** Use `insert()` to add an element at a specific index.

```
restaurants.insert(0, "Pasta Magic")
```

Removing Elements

- **Removing:** Use `remove()` to remove an element by value.

```
restaurants.remove("Pasta Place")
```

- **Removing by Index:** Use `pop()` to remove an element by index.

```
restaurants.pop(0)
```

```
'Pasta Magic'
```

Nested Lists

- **Definition:** Lists containing other lists or tuples.
- **Accessing:** Use nested indexing.

```
restaurant_data = [  
    ["Pasta Place", 4.5, 3],  
    ["Sushi Bar", 4.2, 1]  
]  
print(restaurants[0][1]) # Access the second element of the first list
```

```
a
```

Tuples

- **Definition:** Ordered, immutable collections of items.
- **Creation:** Use parentheses `()`.
- **Immutability:** Once created, cannot be changed.
- **Memory Efficiency:** Use less memory than lists.
- **Use Cases:** Ideal for fixed data (e.g., restaurant location).

```
ratings = (4.5, 3.8, 4.2)  
restaurant_info = ("Pasta Place", "Italian", 2020)
```

Loops

for Loops

- **Definition:** Iterate over a sequence of items.

- **Structure:**

```
for item in sequence:  
    # code to execute for each item
```

- **Example:**

```
treatments = ["Standard Drug", "New Drug A", "New Drug B"]  
for treatment in treatments:  
    print(f"Evaluating efficacy of {treatment}")
```

```
Evaluating efficacy of Standard Drug  
Evaluating efficacy of New Drug A  
Evaluating efficacy of New Drug B
```

Range in for Loops

- **Definition:** Generate a sequence of numbers.
- **Structure:**

```
range(start, stop, step)
```

- **Example:**

```
for phase in range(5): # 0 to 4  
    print(f"Starting Phase {phase + 1}")
```

```
Starting Phase 1  
Starting Phase 2  
Starting Phase 3  
Starting Phase 4  
Starting Phase 5
```

```
for phase in range(1, 5): # 1 to 4  
    print(f"Starting Phase {phase}")
```

```
Starting Phase 1  
Starting Phase 2  
Starting Phase 3  
Starting Phase 4
```

```
for phase in range(1, 5, 2): # 1 to 4, step 2  
    print(f"Starting Phase {phase}")
```

```
Starting Phase 1  
Starting Phase 3
```

break and continue

- **break:** Exit the loop.
- **continue:** Skip the current iteration and continue with the next.

```
efficacy_scores = [45, 60, 75, 85, 90]  
for score in efficacy_scores:  
    if score < 50:  
        continue  
        print(f"Treatment efficacy: {score}%")  
    if score >= 85:  
        break
```

Tuple unpacking

- **Definition:** Assign elements of a tuple to variables.
- **Structure:**
- **Example:**

```
restaurant_info = ("Pasta Place", "Italian", 2020)  
name, cuisine, year = restaurant_info  
print(name)  
print(cuisine)  
print(year)
```

```
Pasta Place  
Italian  
2020
```

while Loops

- **Definition:** Execute code repeatedly as long as a condition is true.
- **Structure:**

```
while condition:  
    # code to execute while condition is True
```

- **Example:**

```
phase = 1  
while phase <= 5:  
    print(f"Starting Phase {phase}")  
    phase += 1
```

```
Starting Phase 1
Starting Phase 2
Starting Phase 3
Starting Phase 4
Starting Phase 5
```

Functions

Basic Function

- **Definition:** Use the `def` keyword.
- **Structure:**

```
def function_name(parameters):
    # code to execute (function body)
    return value # Optional
```

- **Example:**

```
def greet_visitor(name):
    return f"Welcome to the library, {name}!"

greet_visitor("Student")
```

```
'Welcome to the library, Student!'
```

Return Value

- **Definition:** The value returned by a function.
- **Example:**

```
def multiply_by_two(number):
    return number * 2

result = multiply_by_two(5)
print(result)
```

```
10
```

- **Note:** If a function does not return a value, it implicitly returns `None`.

Default Parameters

- **Definition:** Provide default values for function parameters.
- **Structure:**


```
def greet_visitor(name="People"):
    return f"Welcome to the library, {name}!"

print(greet_visitor()) # Calls the function with the default parameter
print(greet_visitor("Tobias")) # Calls the function with a custom parameter
```

Multiple Parameters

- **Definition:** Functions can have multiple parameters.
- **Structure:**

```
def greet_visitor(name, age):
    return f"Welcome to the library, {name}! You are {age} years old."

print(greet_visitor("Tobias", 30))
```

String Methods

- **Definition:** Methods are functions that are called on strings.
- **Structure:**

```
string.method()
```

- **Common String Methods:**
 - `.strip()` - Removes whitespace from start and end
 - `.title()` - Capitalizes first letter of each word
 - `.lower()` - Converts to lowercase
 - `.upper()` - Converts to uppercase
- **Example:**

```
title = "the hitchhikers guide"
print(title.title())
```

```
The Hitchhikers Guide
```

```
title = "  the hitchhikers guide  "
print(title.strip())
```

```
the hitchhikers guide
```

Packages

Standard Libraries

- **Definition:** Libraries that are part of the Python standard library.
- **Access:** Import them using `import`.

```
import math
import random
```

- For long package names, you can use the `as` keyword to create an alias.

```
import random as rd
```

- To call a function from an imported package, use the package name as a prefix.

```
random_number = rd.random()
print(random_number)
```

```
0.7499199021012557
```

Installing Packages

- **Definition:** Install packages using `uv`.

```
uv add package_name
```

- If you are using Miniconda, you can use `conda` instead.

```
conda install package_name
```